arXiv:1706.05806v2 [stat.ML] 8 Nov 2017

SVCCA: Singular Vector Canonical Correlation Analysis for Deep Learning Dynamics and Interpretability

Maithra Raghu,[1,2] **Justin Gilmer,**[1] **Jason Yosinski,**[3] **& Jascha Sohl-Dickstein**[1]
[1]Google Brain [2]Cornell University [3]Uber AI Labs
maithrar@gmail.com, gilmer@google.com, yosinski@uber.com, jaschasd@google.com

Abstract

We propose a new technique, Singular Vector Canonical Correlation Analysis (SVCCA), a tool for quickly comparing two representations in a way that is both invariant to affine transform (allowing comparison between different layers and networks) and fast to compute (allowing more comparisons to be calculated than with previous methods). We deploy this tool to measure the intrinsic dimensionality of layers, showing in some cases needless over-parameterization; to probe learning dynamics throughout training, finding that networks converge to final representations from the bottom up; to show where class-specific information in networks is formed; and to suggest new training regimes that simultaneously save computation and overfit less.

1 Introduction

As the empirical success of deep neural networks ([7, 9, 18]) become an indisputable fact, the goal of better understanding these models escalates in importance. Central to this aim is a core issue of deciphering learned representations. Facets of this key question have been explored empirically, particularly for image models, in [1, 2, 10, 12, 13, 14, 15, 19, 20]. Most of these approaches are motivated by interpretability of learned representations. More recently, [11] studied the similarities of representations learned by multiple networks by finding permutations of neurons with maximal correlation.

In this work we introduce a new approach to the study of network representations, based on an analysis of each neuron's *activation vector* – the scalar outputs it emits on input datapoints. With this interpretation of neurons as vectors (and layers as subspaces, spanned by neurons), we introduce SVCCA, Singular Vector Canonical Correlation Analysis, an amalgamation of Singular Value Decomposition and Canonical Correlation Analysis (CCA) [5], as a powerful method for analyzing deep representations. Although CCA has not previously been used to compare deep representations, it has been used for related tasks such as computing the similarity between modeled and measured brain activity [16], and training multi-lingual word embeddings in language models [3].

The main contributions resulting from the introduction of SVCCA are the following:

1. We ask: is the dimensionality of a layer's learned representation the same as the number of neurons in the layer? *Answer: No.* We show that trained networks perform equally well with a number of directions just a fraction of the number of neurons with no additional training, provided they are carefully chosen with SVCCA (Section 2.1). We explore the consequences for model compression (Section 4.4).
2. We ask: what do deep representation learning dynamics look like? *Answer: Networks broadly converge bottom up.* Using SVCCA, we compare layers across time and find they

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

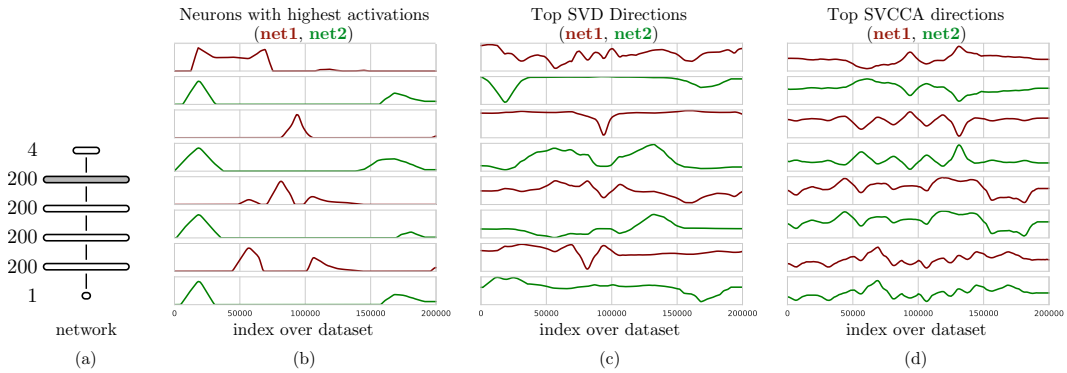

Figure 1: To demonstrate SVCCA, we consider a toy regression task (regression target as in Figure 3). **(a)** We train two networks with four fully connected hidden layers starting from different random initializations, and examine the representation learned by the penultimate (shaded) layer in each network. **(b)** The neurons with the highest activations in net 1 (maroon) and in net 2 (green). The x-axis indexes over the dataset: in our formulation, the *representation* of a neuron is simply its value over a dataset (Section 2). **(c)** The SVD directions — i.e. the directions of maximal variance — for each network. **(d)** The top SVCCA directions. We see that each pair of maroon/green lines (starting from the top) are almost visually identical (up to a sign). Thus, although looking at just neurons (b) seems to indicate that the networks learn very different representations, looking at the SVCCA subspace (d) shows that the information in the representations are (up to a sign) nearly identical.

solidify from the bottom up. This suggests a simple, computationally more efficient method of training networks, *Freeze Training*, where lower layers are sequentially frozen after a certain number of timesteps (Sections 4.1, 4.2).

3. We develop a method based on the discrete Fourier transform which greatly speeds up the application of SVCCA to convolutional neural networks (Section 3).
4. We also explore an interpretability question, of when an architecture becomes sensitive to different classes. We find that SVCCA captures the semantics of different classes, with similar classes having similar sensitivities, and vice versa. (Section 4.3).

Experimental Details Most of our experiments are performed on CIFAR-10 (augmented with random translations). The main architectures we use are a convolutional network and a residual network[1]. To produce a few figures, we also use a toy regression task: training a four hidden layer fully connected network with 1D input and 4D output, to regress on four different simple functions.

2 Measuring Representations in Neural Networks

Our goal in this paper is to analyze and interpret the representations learned by neural networks. The critical question from which our investigation departs is: how should we define the representation of a neuron? Consider that a neuron at a particular layer in a network computes a real-valued function over the network's input domain. In other words, if we had a lookup table of all possible $\text{input} \rightarrow \text{output}$ mappings for a neuron, it would be a complete portrayal of that neuron's functional form.

However, such infinite tables are not only practically infeasible, but are also problematic to process into a set of conclusions. Our primary interest is not in the neuron's response to random data, but rather in how it represents features of a specific dataset (e.g. natural images). Therefore, in this study we take *a neuron's representation to be its set of responses over a finite set of inputs* — those drawn from some training or validation set.

More concretely, for a given dataset $X = \{x_1, \cdots x_m\}$ and a neuron i on layer l, $\boldsymbol{z}_i^l$, we *define* $\boldsymbol{z}_i^l$ to be the *vector* of outputs on X, i.e.

$$\boldsymbol{z}_i^l = (\boldsymbol{z}_i^l(x_1), \cdots, \boldsymbol{z}_i^l(x_m))$$

[1]Convnet layers: `conv-conv-bn-pool-conv-conv-conv-bn-pool-fc-bn-fc-bn-out`. Resnet layers: `conv-(x10 c/bn/r block)-(x10 c/bn/r block)-(x10 c/bn/r block)-bn-fc-out`.

Note that this is a different vector from the often-considered vector of the "representation at a layer of a single input." Here $\boldsymbol{z}_i^l$ is a *single* neuron's response over the entire dataset, not an entire layer's response for a single input. In this view, a neuron's representation can be thought of as a single vector in a high-dimensional space. Broadening our view from a single neuron to the collection of neurons in a layer, the layer can be thought of as the set of neuron vectors contained within that layer. This set of vectors will span some subspace. To summarize:

Considered over a dataset X with m examples, a neuron is a vector in $\mathbb{R}^m$.
A layer is the subspace of $\mathbb{R}^m$ spanned by its neurons' vectors.

Within this formalism, we introduce *Singular Vector Canonical Correlation Analysis* (SVCCA) as a method for analysing representations. SVCCA proceeds as follows:

- **Input:** SVCCA takes as input two (not necessarily different) sets of neurons (typically layers of a network) $l_1 = \{\boldsymbol{z}_1^{l_1}, ..., \boldsymbol{z}_{m_1}^{l_1}\}$ and $l_2 = \{\boldsymbol{z}_1^{l_2}, ..., \boldsymbol{z}_{m_2}^{l_2}\}$
- **Step 1** First SVCCA performs a singular value decomposition of each subspace to get sub-subspaces $l_1' \subset l_1, l_2' \subset l_2$ which comprise of the most important directions of the original subspaces l_1, l_2. In general we take enough directions to explain 99% of variance in the subspace. This is especially important in neural network representations, where as we will show many low variance directions (neurons) are primarily noise.
- **Step 2** Second, compute the Canonical Correlation similarity ([5]) of l_1', l_2': linearly transform l_1', l_2' to be as aligned as possible and compute correlation coefficients. In particular, given the output of step 1, $l_1' = \{\boldsymbol{z'}_1^{l_1}, ..., \boldsymbol{z'}_{m_1'}^{l_1}\}, l_2' = \{\boldsymbol{z'}_1^{l_2}, ..., \boldsymbol{z'}_{m_2'}^{l_2}\}$, CCA linearly transforms these subspaces $\tilde{l}_1 = W_X l_1'$, $\tilde{l}_2 = W_Y l_2'$ such as to maximize the correlations $corrs = \{\rho_1, \dots \rho_{\min(m_1', m_2')}\}$ between the transformed subspaces.
- **Output:** With these steps, SVCCA outputs pairs of aligned directions, $(\tilde{\boldsymbol{z}}_i^{l_1}, \tilde{\boldsymbol{z}}_i^{l_2})$ and how well they correlate, ρ_i. Step 1 also produces intermediate output in the form of the top singular values and directions.

For a more detailed description of each step, see the Appendix. SVCCA can be used to analyse any two sets of neurons. In our experiments, we utilize this flexibility to compare representations across different random initializations, architectures, timesteps during training, and specific classes and layers.

Figure 1 shows a simple, intuitive demonstration of SVCCA. We train a small network on a toy regression task and show each step of SVCCA, along with the resulting very similar representations. SVCCA is able to find hidden similarities in the representations.

2.1 Distributed Representations

An important property of SVCCA is that it is truly a *subspace* method: both SVD and CCA work with $\mathrm{span}(\boldsymbol{z}_1, \dots, \boldsymbol{z}_m)$ instead of being axis aligned to the $\boldsymbol{z}_i$ directions. SVD finds singular vectors $\boldsymbol{z}_i' = \sum_{j=1}^m s_{ij} \boldsymbol{z}_j$, and the subsequent CCA finds a linear transform W, giving orthogonal canonically correlated directions $\{\tilde{\boldsymbol{z}}_1, \dots, \tilde{\boldsymbol{z}}_m\} = \{\sum_{j=1}^m w_{1j} \boldsymbol{z}_j', \dots, \sum_{j=1}^m w_{mj} \boldsymbol{z}_j'\}$. In other words, SVCCA has no preference for representations that are neuron (axes) aligned.

If representations are distributed across many dimensions, then this is a desirable property of a representation analysis method. Previous studies have reported that representations may be more complex than either fully distributed or axis-aligned [17, 21, 11] but this question remains open.

We use SVCCA as a tool to probe the nature of representations via two experiments:

(a) We find that the subspace directions found by SVCCA are disproportionately important to the representation learned by a layer, relative to neuron-aligned directions.

(b) We show that at least some of these directions are distributed across many neurons.

Experiments for (a), (b) are shown in Figure 2 as (a), (b) respectively. For both experiments, we first acquire two different representations, l_1, l_2, for a layer l by training two different random initializations of a convolutional network on CIFAR-10. We then apply SVCCA to l_1 and l_2 to get directions